

Referência da API

Todos os caminhos são relativos à raiz da aplicação. Os corpos são JSON, em UTF-8. Para o contexto de como as coisas funcionam, ver [DESENVOLVIMENTO.md](#).

AUTENTICAÇÃO

A autenticação é **por sessão**, com cookie, não por token. O fluxo de um cliente é:

1. Fazer um `GET` a qualquer página (por exemplo `/login.html`) para receber o cookie `XSRF-TOKEN`.
2. `POST /api/auth/login`, que devolve o cookie de sessão `JSESSIONID`.
3. Enviar os dois cookies em todos os pedidos seguintes.

CSRF

Todos os pedidos que **não** sejam `GET` têm de levar o valor do cookie `XSRF-TOKEN` no cabeçalho `X-XSRF-TOKEN`. Sem ele a resposta é `403` com a mensagem "Pedido rejeitado por falta de token de segurança".

```
curl -s -c cookies -o /dev/null http://localhost:8080/login.html
TOKEN=$(grep XSRF-TOKEN cookies | awk '{print $NF}')
```

```
curl -b cookies -c cookies \
  -H "Content-Type: application/json" -H "X-XSRF-TOKEN: $TOKEN" \
  -X POST http://localhost:8080/api/auth/login \
  -d '{"email":"gestor@exemplo.pt","password":"apasswordtoda"}'
```

Quem pode o quê

CAMINHO	REQUISITO
<code>/api/auth/login</code> , <code>/registo</code> , <code>/registo-treinador</code> , <code>/estado</code>	público
<code>/api/auth/recuperar</code> , <code>/api/auth/recuperar/confirmar</code>	público
<code>/actuator/health</code>	público
<code>/api/admin/**</code>	papel <code>ADMIN</code>
<code>POST /api/ligas</code>	permissão <code>PODE_CRIAR_LIGAS</code>
tudo o resto	sessão iniciada

Além disto, quase tudo é isolado pelo dono: uma liga que não seja tua responde `404`, não `403`, para não confirmar que existe.

ERROS

Todos os erros têm o mesmo formato:

```
{ "status": 409, "mensagem": "Já existe uma equipa com este nome nesta liga." }
```

ESTADO	QUANDO
400	dados inválidos, campo em falta, JSON ilegível, id mal formado
401	sem sessão, credenciais erradas, ou conta desactivada a meio da sessão
403	sem permissão, convite inválido, ou falta o token CSRF
404	não existe, ou não é teu
409	regra de negócio violada, ou duas alterações em simultâneo
413	imagem acima do limite
500	erro inesperado (a mensagem interna nunca é exposta)

A distinção que interessa: **400 é o pedido estar mal formado**, **409 é o pedido estar bem formado mas não poder ser feito agora** (fechar uma jornada já fechada, inscrever uma equipa com nome repetido, desempatar uma jornada que não está em desempate).

AUTENTICAÇÃO E CONTA

POST /api/auth/login

Público.

```
{ "email": "gestor@exemplo.pt", "password": "apasswordtoda" }
```

200 devolve **GestorDto**. **401** se as credenciais não servirem ou a conta estiver desactivada (a mensagem é a mesma nos dois casos, de propósito).

POST /api/auth/registo

Público. Cria conta de gestor a partir de um convite de administrador e inicia sessão logo a seguir.

```
{ "codigo": "...", "nome": "João", "email": "joao@exemplo.pt", "password": "apasswordtoda" }
```

201 devolve **GestorDto**. Password com 10 caracteres no mínimo.

POST /api/auth/registo-treinador

Público. Igual ao anterior, mas a partir de um convite de treinador. A conta criada fica com `podeCriarLigas: false`.

POST /api/auth/treinador/ligar

Liga um convite de treinador à conta com sessão aberta, sem criar conta nova.

```
{ "codigo": "..."} 
```

204 . Dá 409 se aquele treinador já tiver conta.

GET /api/auth/estado

Devolve o `GestorDto` de quem tem sessão, ou 401. É por aqui que o frontend decide se mostra a aplicação ou a página de entrada.

POST /api/auth/password

```
{ "atual": "apasswordantiga", "nova": "apasswordnova"} 
```

204 . **A sessão é invalidada a seguir**, de propósito: quem mudou volta a entrar, e qualquer sessão aberta noutro sítio deixa de servir.

POST /api/auth/recuperar

```
{ "email": "joao@exemplo.pt"} 
```

204 **sempre**, exista a conta ou não. É deliberado: se distinguísse os dois casos, este endereço passava a ser uma forma de descobrir quem tem conta na aplicação, sem sequer precisar de tentar uma password. Um email mal formado também dá 204.

Envia uma mensagem com um link para `/nova-password.html?codigo=...`. O código tem 24 bytes aleatórios, vale uma hora e serve uma vez. Na base de dados fica só o SHA-256 dele.

Limite de **três pedidos por conta por hora**. Acima disso continua a responder 204, mas não envia nada.

POST /api/auth/recuperar/confirmar

```
{ "codigo": "o-codigo-do-link", "password": "apasswordnova"} 
```

204 . 400 se o código for inventado, já usado ou expirado (a mensagem é a mesma nos três casos), e 400 se a password tiver menos de 10 caracteres. Uma password recusada **não gasta o código**: o link continua a servir.

Não abre sessão: quem chega aqui não tinha nenhuma. Corta **todas as sessões abertas** dessa conta, incluindo as de outros dispositivos.

POST /api/auth/logout

204 .

LIGAS

GET /api/ligas

Lista as ligas de quem tem sessão, por nome. Devolve `LigaDto[]` .

POST /api/ligas

Requer `PODE_CRIAR_LIGAS` .

```
{ "nome": "Liga dos Pobres 25/26", "maxEquipas": 20 }
```

201 devolve `LigaDto` . `maxEquipas` entre 1 e 45.

GET /api/ligas/{ligaId}

O detalhe completo: liga, equipas, jornadas, classificação e pote. É o pedido que o frontend faz sempre que a liga muda.

```
{
  "liga": { "id": "...", "nome": "...", "estado": "ATIVA", "maxEquipas": 20,
    "totalEquipas": 8, "equipasAtivas": 7, "totalJornadas": 5,
    "temLogo": true },
  "equipas": [ { "id": "...", "nome": "...", "treinador": "...", "estado": "ATIVA" } ],
  "jornadas": [ { "id": "...", "numero": 1, "estado": "FECHADA", "tipo": "TREINO",
    "treino": true,
    "resultados": [ { "id": "...", "equipaId": "...", "equipa": "...",
      "pontuacao": 5, "posicao": 1 } ] } ],
  "classificacao": [ { "posicao": 1, "equipaId": "...", "equipa": "...",
    "treinador": "...", "estado": "ATIVA", "pontos": 26 } ],
  "pote": { "total": 45.00, "pago": 15.00, "porPagar": 30.00 }
}
```

As jornadas vêm por ordem cronológica real: todas as de treino antes das oficiais.

GET /api/ligas/{ligaId}/classificacao

Só a classificação, em `ClassificacaoDto[]` . Empates de pontos são ordenados por nome.

POST /api/ligas/{ligaId}/terminar

Fecha a liga. Sem corpo. 409 se já estivesse terminada. Não tem volta.

Logo

POST /api/ligas/{ligaId}/logo recebe multipart/form-data com o campo ficheiro . Máximo 1 MB. O formato é decidido pelos primeiros bytes do ficheiro, não pelo Content-Type : PNG, JPEG ou WEBP. 413 se for grande de mais, 400 se não for imagem reconhecida, 409 se a liga estiver terminada.

GET devolve os bytes com o Content-Type correcto. DELETE remove.

EQUIPAS

POST /api/ligas/{ligaId}/equipas

```
{ "nome": "Bairro FC", "treinador": "João Azul" }
```

201 devolve EquipaDto . Cria também o Treinador , sem conta.

Se a liga tiver regra com valor de inscrição, **a inscrição é cobrada aqui**.

409 se a liga estiver terminada, cheia, ou já tiver uma equipa com esse nome (a comparação ignora maiúsculas).

POST /api/ligas/{ligaId}/equipas/{equipaId}/desistencia

Marca a equipa como desistente. Sem corpo. Deixa de contar para jornadas seguintes, mas **a dívida dela mantém-se**. 409 se já não estivesse activa.

Convites de treinador

POST /api/ligas/{ligaId}/equipas/{equipaId}/convites-treinador

```
{ "diasValidade": 30 }
```

diasValidade pode ser null para não expirar. 201 devolve o convite **com o código**, que só aqui é visível.

409 se aquele treinador já tiver conta.

DELETE .../convites-treinador/{conviteId} revoga um convite por usar.

JORNADAS

GET /api/ligas/{ligaId}/jornadas

JornadaDto[] , por ordem cronológica.

POST /api/ligas/{ligaId}/jornadas

Abre a próxima. Sem corpo. 201 devolve JornadaDto com o número e o tipo atribuídos.

409 se já existir uma jornada por fechar (incluindo uma em desempate) ou se a liga estiver terminada.

GET /api/ligas/{ligaId}/jornadas/{jornadaId}

JornadaDto .

PUT /api/ligas/{ligaId}/jornadas/{jornadaId}/resultados

Insere ou actualiza a pontuação de uma equipa.

```
{ "equipaId": "...", "pontuacao": 7 }
```

pontuacao é obrigatória e não pode ser negativa; um decimal onde se espera um inteiro é recusado com 400 , não truncado.

409 se a jornada estiver fechada, em desempate, ou se a equipa for desistente.

POST /api/ligas/{ligaId}/jornadas/{jornadaId}/fechar

Sem corpo. Atribui posições por pontuação decrescente.

Duas respostas possíveis, ambas 200 :

- "estado": "FECHADA" se não houve empates. Pode ter fechado um bloco de dívida.
- "estado": "DESEMPATE" se ficaram equipas empatadas. **Não foi cobrado nada.** Resolve com o endpoint seguinte.

409 se já estiver fechada, se já estiver à espera de desempate, ou se não houver resultados nenhuns.

POST /api/ligas/{ligaId}/jornadas/{jornadaId}/desempate

```
{ "ordem": ["equipaId-1", "equipaId-2"] }
```

A lista tem de conter **exactamente** as equipas empatadas, cada uma uma vez, da melhor para a pior. Podem ser vários grupos de empate numa lista só: a pontuação manda na ordenação e a ordem dada só decide entre iguais.

200 devolve a jornada já fechada, com posições sequenciais. Só depois disto é que o bloco de dívida é cobrado.

400 se a lista não corresponder ao conjunto empatado. 409 se a jornada não estiver à espera de desempate.

DÍVIDAS

GET /api/ligas/{ligaId}/regra-divida

RegraDividaDto, ou 404 se a liga ainda não tiver regra. 404 **aqui é normal**, não é erro: é o estado de uma liga que cobra tudo à mão.

PUT /api/ligas/{ligaId}/regra-divida

Cria ou substitui. Todos os campos são obrigatórios.

```
{ "valorInscricao": 15.00, "valorInicial": 0.00, "incremento": 0.50,
  "equipasPorEscalao": 5, "valorMaximo": 2.50, "jornadasPorBloco": 5 }
```

equipasPorEscalao e jornadasPorBloco são inteiros com mínimo 1, os valores não podem ser negativos e valorMaximo não pode ser inferior a valorInicial.

Pode ser alterada a meio da época. As jornadas já cobradas não são recalculadas.

GET /api/ligas/{ligaId}/dividas?estado=PENDENTE

Dívidas da liga, filtradas pelo estado da dívida (PENDENTE ou RESOLVIDA). Omitir estado equivale a PENDENTE.

Este filtro é sobre a dívida da equipa, não sobre cada bloco. Para somar dinheiro, usa o pote do detalhe da liga.

GET /api/ligas/{ligaId}/equipas/{equipaId}/divida

```
{
  "id": "...", "equipaId": "...", "equipaNome": "Bairro FC",
  "ligaId": "...", "ligaNome": "Liga dos Pobres",
  "estado": "PENDENTE", "totalPendente": 17.50,
  "blocos": [ { "id": "...", "numeroBloco": 1, "tipo": "INSCRICAO",
    "valor": 15.00, "estado": "RESOLVIDA",
    "criadoEm": "...", "resolvidoEm": "..." } ]
}
```

Uma equipa que ainda não tenha sido cobrada devolve "estado": "SEM_DIVIDA", id a null e a lista vazia, em vez de 404.

POST /api/ligas/{ligaId}/equipas/{equipaId}/divida/blocos

Lança um bloco à mão.

```
{ "valor": 2.50 }
```

`201` devolve `BlocoDividaDto`. O valor não pode ser negativo. Um bloco de zero nasce já resolvido.

`POST .../divida/blocos/{blocoId}/pagar`

Marca um bloco como pago. Sem corpo. `409` se já estivesse pago.

`POST .../divida/pagar`

Marca tudo o que a equipa deve. Sem corpo. `404` se a equipa não tiver dívida.

VISTAS DO TREINADOR

Só de leitura. A autorização não é por dono da liga, é por a conta treinar alguma equipa lá dentro.

`GET /api/minhas-ligas`

`LigaDto[]` das ligas onde a conta treina alguma equipa, sem repetições.

`GET /api/minhas-ligas/{ligaId}`

O mesmo `LigaDetalheDto` do gestor, incluindo o pote, que é um valor agregado da liga. `404` se a conta não treinar lá nada.

`GET /api/minhas-dividas`

```
{ "equipas": [ /* DividaDto por equipa, ordenadas por liga e nome */ ],  
  "totalGeral": 32.50 }
```

Parte das equipas que a conta treina, não das dívidas, por isso uma equipa ainda sem nada cobrado aparece na mesma, a dever zero.

ADMINISTRAÇÃO

Tudo abaixo requer papel `ADMIN`.

`GET /api/admin/gestores`

`GestorAdminDto[]`, por data de criação.

`PATCH /api/admin/gestores/{gestorId}`

Envia só os campos a mudar; pelo menos um é obrigatório.

```
{ "ativo": false, "papel": "ADMIN", "podeCriarLigas": true, "email": "novo@exemplo.pt" }
```

409 ao tentar alterar a própria conta, ao desactivar/despromover o último administrador activo, ou ao pôr um email que já é de outra conta.

O **email** existe para desenrascar quem se enganou a escrevê-lo no registo: sem isto essa conta não recebe o link de recuperação e fica presa, porque só o próprio muda o email e o próprio já não consegue entrar. Muda-lo **corta as sessões abertas** dessa conta.

Uma alteração aqui tem efeito **no pedido seguinte** da pessoa afectada, mesmo que ela esteja com sessão aberta.

GET /api/admin/convites

ConviteDto[], do mais recente para o mais antigo. O **codigo** só vem preenchido enquanto o convite estiver por usar.

POST /api/admin/convites

```
{ "nota": "Ricardo, da Rotunda", "diasValidade": 30 }
```

Ambos opcionais (**null** para sem nota e sem prazo). **201** devolve o convite com o código, que só aqui é visível.

DELETE /api/admin/convites/{conviteId}

Revoga. **409** se já tiver sido usado ou revogado.

ESTADOS

ENUM	VALORES
EstadoLiga	ATIVA , DESATIVADA
EstadoEquipa	ATIVA , DESISTENTE
EstadoJornada	ABERTA , DESEMPATE , FECHADA
EstadoJornadaTreino (campo tipo)	TREINO , OFICIAL
EstadoDivida	PENDENTE , RESOLVIDA (e SEM_DIVIDA , só na resposta)
TipoBloco	INSCRICAO , PERIODO
PapelGestor	GESTOR , ADMIN